

Roteiro Aula Prática



ROTEIRO DE AULA PRÁTICA

NOME DA DISCIPLINA: ALGORITMOS E ESTRUTURA DE DADOS AVANÇADO

Unidade: U1_ FUNDAMENTOS DE ALGORITMOS

Aula: A4_ NOÇÕES DE ORDENAÇÃO

OBJETIVOS

Definição dos objetivos da aula prática:

Implementar e comparar diferentes algoritmos de ordenação em um cenário de aplicação realista. O objetivo é entender a eficiência e a aplicabilidade de cada algoritmo em diferentes situações.

SOLUÇÃO DIGITAL

Computador com acesso à Internet para uso do Google Colab

O Google Colab, ou Colaboratory, é uma plataforma gratuita baseada na nuvem oferecida pelo Google. Ela fornece um ambiente de notebook interativo e colaborativo que permite a criação e execução de código diretamente no navegador, sem a necessidade de configurar ou instalar qualquer software no seu computador.

<https://colab.google/>

PROCEDIMENTOS PRÁTICOS

Procedimento/Atividade nº 1

Atividade proposta:

Você trabalha em uma empresa de e-commerce e precisa ordenar uma lista de produtos com base em diferentes critérios para melhorar a experiência do usuário e a eficiência do sistema de recomendação. A lista de produtos inclui informações como preço, avaliação dos usuários, data de adição ao catálogo e categoria.

Procedimentos para a realização da atividade:

1. Preparação dos Dados:

- Crie uma classe `Produto` com os seguintes atributos:
 - `nome`: string
 - `preco`: float

- ``avaliacao``: float (0 a 5)
- ``data_adicao``: datetime
- ``categoria``: string

2. Geração de Dados:

- Escreva um script para gerar uma lista de 1000 produtos aleatórios. Utilize bibliotecas como ``random`` e ``datetime`` para preencher os atributos de cada produto.

3. Implementação de Algoritmos de Ordenação:

- Implemente os seguintes algoritmos de ordenação:
 - Bubble Sort
 - Quick Sort
 - Merge Sort
 - Heap Sort

4. Critérios de Ordenação:

- Implemente funções de ordenação para os seguintes critérios:
 - Por preço (ascendente e descendente)
 - Por avaliação (ascendente e descendente)
 - Por data de adição (mais recente primeiro e mais antigo primeiro)
 - Por categoria (ordem alfabética)

5. Comparação de Desempenho:

- Meça e compare o tempo de execução de cada algoritmo para cada critério de ordenação utilizando a biblioteca ``time``.

6. Análise de Resultados:

- Escreva um relatório discutindo a eficiência de cada algoritmo de ordenação nos diferentes critérios. Considere a complexidade temporal de cada algoritmo e como eles se comportam com os dados gerados.

Dicas

- Utilize a função ``sorted()`` do Python para verificar a corretude das suas implementações.
- A biblioteca ``time`` pode ser utilizada para medir o tempo de execução de um bloco de código.
- A biblioteca ``datetime`` pode ajudar na manipulação de datas.
- Para visualização, você pode utilizar gráficos de barras para mostrar o tempo de execução de cada algoritmo em diferentes cenários. A biblioteca ``matplotlib`` pode ser útil aqui.

Código Inicial:

```
import random
import datetime
import time

class Produto:
    def __init__(self, nome, preco, avaliacao, data_adicao, categoria):
        self.nome = nome
        self.preco = preco
        self.avaliacao = avaliacao
        self.data_adicao = data_adicao
        self.categoria = categoria

    def __repr__(self):
        return f"{self.nome}: {self.preco}, {self.avaliacao}, {self.data_adicao}, {self.categoria}"

def gerar_produtos(n):
    nomes = ["Produto" + str(i) for i in range(n)]
    precos = [round(random.uniform(10, 1000), 2) for _ in range(n)]
    avaliacoes = [round(random.uniform(0, 5), 2) for _ in range(n)]
    datas = [datetime.datetime.now() - datetime.timedelta(days=random.randint(0, 365)) for _ in range(n)]
    categorias = ["Categoria" + str(random.randint(1, 5)) for _ in range(n)]

    produtos = [Produto(nomes[i], precos[i], avaliacoes[i], datas[i], categorias[i]) for i in range(n)]
    return produtos

# Exemplo de geração de produtos
produtos = gerar_produtos(1000)
for produto in produtos[:10]: # Mostrar os 10 primeiros produtos
    print(produto)
```

Checklist:

- Preparação dos Dados
- Geração de Dados
- Implementação de Algoritmos de Ordenação

- Critérios de Ordenação
- Comparação de Desempenho
- Análise de Resultados

RESULTADOS

Resultados de Aprendizagem:

Espera-se que o aluno seja capaz de entender a implementação e a análise dos principais algoritmos de ordenação aplicados a um cenário realista, proporcionando uma compreensão prática e teórica sólida sobre a eficiência dos diferentes métodos de ordenação.

ESTUDANTE, VOCÊ DEVERÁ ENTREGAR

Descrição orientativa sobre a entrega da comprovação da aula prática:

- Para comprovar a realização da atividade, é necessário entregar um arquivo com os códigos criados e um PDF com o relatório de análise.

Unidade: U2_ ÁRVORES

Aula: A1_ ÁRVORES AVL

OBJETIVOS

Definição dos objetivos da aula prática:

Entender os conceitos de balanceamento e rotação em árvores binárias de busca implementando uma Árvore AVL em Python, incluindo inserção, remoção e busca de nós, além de garantir que a árvore permaneça balanceada após cada operação.

SOLUÇÃO DIGITAL

Computador com acesso à Internet para uso do Google Colab

O Google Colab, ou Colaboratory, é uma plataforma gratuita baseada na nuvem oferecida pelo Google. Ela fornece um ambiente de notebook interativo e colaborativo que permite a criação e execução de código diretamente no navegador, sem a necessidade de configurar ou instalar qualquer software no seu computador.

<https://colab.google/>

PROCEDIMENTOS PRÁTICOS

Procedimento/Atividade nº 1

Atividade proposta:

Você trabalha em uma empresa de tecnologia que está desenvolvendo um sistema de gerenciamento de dados. Para otimizar as operações de busca, inserção e remoção, você foi designado para implementar uma Árvore AVL que manterá os dados balanceados.

Procedimentos para a realização da atividade:

1. Definição da Estrutura da Árvore AVL:

- Crie uma classe `Node` para representar cada nó da árvore.
- Crie uma classe `AVLTree` para gerenciar as operações na árvore.

2. Implementação de Operações Básicas:

- Implementar a inserção de nós na árvore AVL.
- Implementar a remoção de nós da árvore AVL.
- Implementar a busca de nós na árvore AVL.

3. Balanceamento da Árvore:

- Implementar as rotações à esquerda e à direita para manter a árvore balanceada.
- Garantir que, após cada inserção e remoção, a árvore permaneça uma AVL válida.

4. Testes de Validação:

- Escreva testes para validar a inserção, remoção e busca em diferentes cenários.
- Testar casos de borda como inserção de nós em ordem ascendente ou descendente para verificar o balanceamento.

5. Visualização da Árvore:

- Implementar uma função para imprimir a árvore de forma que seja fácil visualizar sua estrutura e balanceamento.

Dicas

- Utilize a propriedade de altura dos nós para ajudar no balanceamento.
- Uma árvore AVL é uma árvore binária de busca onde a diferença de altura entre as subárvores esquerda e direita de qualquer nó é no máximo 1.
- As rotações (simples e duplas) são cruciais para manter a árvore balanceada.

Checklist:

- Definição da Estrutura da Árvore AVL:
- Implementação de Operações Básicas:
- Balanceamento da Árvore:
- Testes de Validação:
- Visualização da Árvore:

RESULTADOS

Resultados de Aprendizagem:

Espera-se que o aluno seja capaz de entender a implementação de uma Árvore AVL em Python com operações de inserção, remoção e busca, além do balanceamento automático, demonstrando habilidade prática.

ESTUDANTE, VOCÊ DEVERÁ ENTREGAR

Descrição orientativa sobre a entrega da comprovação da aula prática:

Para comprovar a realização da atividade, é necessário entregar um relatório em PDF com:

- Códigos criados
- Prints de tela com os resultados da execução
- Um breve relatório de análise.

Unidade: U3_ GRAFOS

Aula: A3_ CAMINHOS MÍNIMOS

OBJETIVOS

Definição dos objetivos da aula prática:

Compreender os conceitos de grafos, algoritmos de busca de caminhos mínimos e estruturas de dados como listas de adjacência e filas de prioridade.

SOLUÇÃO DIGITAL

Computador com acesso à Internet para uso do Google Colab

O Google Colab, ou Colaboratory, é uma plataforma gratuita baseada na nuvem oferecida pelo Google. Ela fornece um ambiente de notebook interativo e colaborativo que permite a criação e execução de código diretamente no navegador, sem a necessidade de configurar ou instalar qualquer software no seu computador.

PROCEDIMENTOS PRÁTICOS

Procedimento/Atividade nº 1

Atividade proposta:

Você está desenvolvendo um sistema de navegação para uma aplicação de mapas. Para encontrar a rota mais curta entre dois pontos, você precisa implementar o algoritmo de Dijkstra.

Procedimentos para a realização da atividade:

1. Definição da Estrutura do Grafo:

- Crie uma classe **Graph** para representar o grafo usando uma lista de adjacência.
- Cada aresta do grafo deve ter um peso associado.

2. Implementação do Algoritmo de Dijkstra:

- Implemente o algoritmo de Dijkstra para encontrar o caminho mais curto a partir de um nó de origem para todos os outros nós do grafo.
- Utilize uma fila de prioridade (min-heap) para otimizar a escolha do próximo nó com a menor distância.

3. Função para Encontrar o Caminho Mínimo:

- Implemente uma função que, dado um nó de origem e um nó de destino, retorne o caminho mínimo e a distância mínima entre esses nós.

4. Testes de Validação:

- Escreva testes para validar o algoritmo com diferentes grafos e nós de origem e destino.
- Teste casos de borda, como grafos desconectados ou nós sem arestas.

5. Visualização do Caminho:

- Implemente uma função para imprimir o caminho mínimo de forma legível.

Dicas

- Utilize um dicionário para representar o grafo onde as chaves são os nós e os valores são listas de tuplas (vizinho, peso).
- A fila de prioridade pode ser implementada usando o módulo `heapq` do Python.

- Mantenha um dicionário de distâncias mínimas e um dicionário de predecessores para reconstruir o caminho

RESULTADOS

Resultados de Aprendizagem:

Espera-se que o aluno seja capaz de entender a implementação de um algoritmo que encontra caminhos mínimos em grafos.

ESTUDANTE, VOCÊ DEVERÁ ENTREGAR

Descrição orientativa sobre a entrega da comprovação da aula prática:

Para comprovar a realização da atividade, é necessário entregar um relatório em PDF com:

- Códigos criados
- Prints de tela com os resultados da execução
- Um breve relatório explicando todo o procedimento realizado.

Unidade: U4_ COMPRESSÃO DE DADOS E OUTRAS ESTRUTURAS

Aula: A1_ HEAP

OBJETIVOS

Definição dos objetivos da aula prática:

Aprender a construir uma lista de prioridade, inserir e remover elementos no heap, e alterar a prioridade de elementos existentes.

SOLUÇÃO DIGITAL

Infraestrutura mínima necessária para execução.

Computador com acesso à Internet para uso do Google Colab

O Google Colab, ou Colaboratory, é uma plataforma gratuita baseada na nuvem oferecida pelo Google. Ela fornece um ambiente de notebook interativo e colaborativo que permite a criação e execução de código diretamente no navegador, sem a necessidade de configurar ou instalar qualquer software no seu computador.

<https://colab.google/>

PROCEDIMENTOS PRÁTICOS

Procedimento/Atividade nº 1

Atividade proposta:

Você deverá implementar uma lista de prioridade usando um heap (min-heap) em Python.

Procedimentos para a realização da atividade:

1. Construção da Lista de Prioridade: Construa a classe `PriorityQueue``
 - Implemente uma lista de prioridade usando um min-heap.
 - crie uma função para inicializar a lista de prioridade.
 - Implemente funções para inserir elementos na lista de prioridade.
 - Implemente funções para remover o elemento com a menor prioridade.
 - Implemente uma função para alterar a prioridade de um elemento existente.
2. Teste da Lista de Prioridade: Implemente a função `test_priority_queue`` para testar a lista de prioridade
 - Inicialize uma lista de prioridade.
 - Adicione tarefas com diferentes prioridades e exiba a lista.
 - Insira novas tarefas na lista de prioridade.
 - Remova a tarefa com a menor prioridade e exiba a lista após cada remoção.
 - Altere a prioridade de uma tarefa existente e exiba a lista.
 - Verifique os resultados e assegure-se de que as operações de inserção, remoção e alteração de prioridade funcionam conforme esperado.

Dicas

- Use a estrutura de min-heap do módulo `heapq`` para gerenciar a lista de prioridade.
- Mantenha um dicionário (`entry_finder``) para rastrear os itens na lista e facilitar a alteração de prioridade.
- Use um contador (`counter``) para diferenciar entre tarefas com a mesma prioridade.

RESULTADOS

Resultados de Aprendizagem:

Espera-se que o aluno seja capaz de entender a implementação de um algoritmo de lista de prioridades com heap.

ESTUDANTE, VOCÊ DEVERÁ ENTREGAR

Descrição orientativa sobre a entrega da comprovação da aula prática:

Para comprovar a realização da atividade, é necessário entregar um relatório em PDF com:

- Códigos criados
- Prints de tela com os resultados da execução
- Um breve relatório explicando todo o procedimento realizado.